Proposta Agente IA Educacional Local

https://gemini.google.com/app/96e2c4bba98a2d8a

User prompt: criar uma proposta para a cadeira de PROJETO DE DESENVOLVIMENTO DE SOFTWARE, minha ideia criar um agente ou configurar um agente de IA que rode localmente sem conexão externa que auxilie em atividades educativas. o problema e fazer ele rodar em um ryzen 7 5700U com 32 giga de memoria

Response: O seu hardware é mais do que suficiente para rodar um agente local, desde que a arquitetura escolha modelos otimizados para execução via CPU e RAM DDR4. O grande diferencial da sua máquina são os **32 GB de RAM**, que eliminam o gargalo de memória, enquanto o **Ryzen 7 5700U** (8 núcleos / 16 threads) lida muito bem com inferência via instruções AVX2.

Aqui está uma estrutura pronta para você apresentar na cadeira de **Projeto de Desenvolvimento de Software**.

Proposta de Projeto de Software (PDS)

Título do Projeto: Agente Local Interativo para Apoio Educacional (EduAgent-Local)

Natureza do Projeto: Software de Assistência Educacional / Inteligência Artificial Distribuída Localmente

1. Contexto e Problema

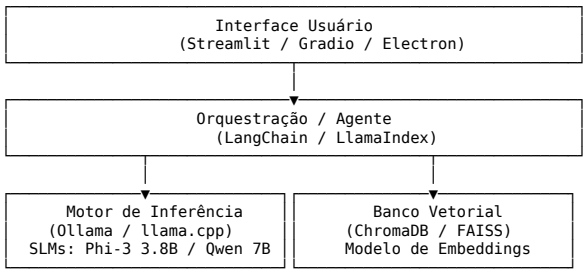
A adoção de assistentes virtuais de IA no ambiente educacional enfrenta barreiras como dependência de conexão com a internet, custos com APIs e riscos à privacidade dos dados de estudantes/professores. Criar uma solução totalmente *offline* e local resolve essa limitação, permitindo o uso em escolas com infraestrutura de rede limitada ou por usuários que exigem privacidade total dos seus documentos de estudo.

2. Objetivos

- **Objetivo Geral:** Desenvolver uma aplicação de agente educacional 100% offline capaz de processar documentos de estudo (PDFs, textos) e interagir em linguagem natural.
- **Objetivos Específicos:**
  - Configurar e otimizar um motor de inferência LLM para execução via CPU/RAM.
  - Implementar uma arquitetura RAG (*Retrieval-Augmented Generation*) para consulta a materiais pedagógicos sem conexão externa.
  - Criar uma interface gráfica simples e responsiva voltada para estudantes e professores.

3. Solução Técnica e Arquitetura

Para obter um desempenho satisfatório (de 6 a 12 tokens/segundo), o sistema fará uso de modelos **quantizados** (formato GGUF / quantização 4-bit) executados via motor nativo em C++ (llama.cpp).



Especificação de Modelos Sugeridos

Com 32 GB de RAM, você tem espaço de sobra, mas deve optar por **SLMs (Small Language Models)** para manter a velocidade de resposta rápida no processamento por CPU:

- **Para Chat e Resumos:** `Phi-3-Mini` (3.8B) ou `Qwen2.5-7B` (quantizado em 4-bits - GGUF).
- **Para Busca e Leitura de PDFs:** Modelo de embedding leve como `all-MiniLM-L6-v2` ou `bge-small-pt` para busca vetorial local rápida.

4. Viabilidade Técnica no Hardware (Ryzen 7 5700U + 32GB RAM)

- **Uso de Memória RAM:** Um modelo 7B quantizado em 4-bit ocupa apenas cerca de **4.5 GB a 6 GB de RAM**. O banco vetorial local e o sistema gastam menos de 2 GB. Sobram mais de 20 GB de RAM no seu notebook, garantindo estabilidade total.
- **Capacidade de Processamento:** O processador Ryzen 7 5700U possui 16 threads. Alocando de 6 a 8 threads para o motor `llama.cpp` / `Ollama`, o sistema atingirá uma velocidade fluida de geração contínua.
- **Estratégia de Otimização no Código:**
  - Configurar o parâmetro de contextos (*context window*) para números razoáveis (ex: 2048 a 4096 tokens) para economizar ciclos do processador.
  - Uso de *Quantização GGUF (Q4_K_M)* para máxima eficiência no processamento via AVX2.

5. Escopo e Funcionalidades do Agente Educacional

1. **Leitura e Fichamento de Documentos (RAG):** O aluno insere um PDF de uma aula ou livro e faz perguntas diretamente sobre aquele conteúdo.
2. **Gerador de Exercícios e Simulados:** O agente formula questões de múltipla escolha com base no texto lido e fornece feedback imediato.
3. **Tutor Explicativo:** Explicador de conceitos complexos usando metáforas e linguagem adaptada para níveis de ensino configuráveis (Fundamental, Médio, Superior).

6. Cronograma de Desenvolvimento (Proposta para a Cadeira)

- **Semana 1-2:** Levantamento de requisitos e configuração do ambiente local (Docker, Ollama, bibliotecas Python).
- **Semana 3-4:** Desenvolvimento do pipeline de RAG (extração de texto de PDFs, geração de embeddings e armazenamento em banco vetorial local).
- **Semana 5-6:** Integração do agente, prompts do sistema (*system prompts*) educacionais e testes de desempenho no Ryzen 5700U.
- **Semana 7-8:** Criação da interface do usuário (Gradio/Streamlit) e testes com usuários reais (alunos/professores).
- **Semana 9:** Validação de métricas locais (tempo de resposta, consumo de memória, taxa de acerto do RAG) e entrega da documentação final do projeto.

Sources

1. AMD Ryzen 7 5700U - CPU Benchmarks
2. AMD 5700G for experimenting with local LLMs? : r/LocalLLaMA - Reddit

---

User prompt: ok no fim escolhi uma outra proposta pro tcc. mas to afim de rodar uma ia no meu linux mint. como devo iniciar

Response: Para rodar IA localmente no Linux Mint de forma simples e rápida, o **Ollama** é o melhor ponto de partida. Ele gerencia o download dos modelos, otimiza o uso de CPU/RAM para o seu Ryzen 7 5700U e cria um servidor local em pouquíssimos passos.

Passo 1: Instalar o Ollama

Abra o terminal (`Ctrl` + `Alt` + T) e rode o comando oficial:

```
curl -fsSL https://ollama.com/install.sh | sh
```

O script vai baixar o binário e configurar o serviço no sistema automaticamente. Para verificar se está rodando:

```
ollama --version
```

Passo 2: Baixar e testar um modelo leve

Como seu processador processa o modelo via CPU, você precisa de modelos pequenos e eficientes (de 3B a 8B parâmetros em quantização 4-bit/GGUF). Com seus 32 GB de RAM, estes modelos vão voar:

- **Para conversas em português e respostas rápidas:** `Qwen 2.5 (7B)`

  ```
  ollama run qwen2.5:7b
  ```

- **Para raciocínio e tarefas gerais (muito rápido):** `Phi-3.5 (3.8B)`

  ```
  ollama run phi3.5
  ```

Assim que o download terminar, o terminal abrirá um chat direto com a IA. Basta digitar suas mensagens. Para sair do chat no terminal, digite `/bypass` ou digite `/quit`.

Passo 3: Colocar uma Interface Gráfica (Estilo ChatGPT)

Conversar no terminal funciona, mas ter uma interface web torna a experiência muito melhor. A opção mais leve e moderna é a **Open WebUI**.

A forma mais simples de rodar no Linux Mint é via **Docker**.

1. **Instale o Docker** (caso ainda não tenha):

   ```
   sudo apt update
   sudo apt install docker.io -y
   sudo usermod -aG docker $USER
   ```

   (Depois de rodar o comando acima, encerre a sessão do Linux Mint e entre novamente para aplicar a permissão).

2. **Rode o container da Open WebUI**:

   ```
   docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway --name open-webui --restart always ghcr.io/open-webui/open-webui:main
   ```

3. Abra o navegador e acesse: **`http://localhost:3000`**

Você criará uma conta local rápida e já poderá selecionar os modelos baixados no Ollama, anexar arquivos e gerar respostas em uma interface idêntica à do ChatGPT.

Dica de Otimização para o Ryzen 5700U

No Linux Mint, você pode garantir que o Ollama use todos os seus 16 threads de CPU ajustando uma variável de ambiente se sentir que a resposta está demorando.

Basta criar/editar a configuração do serviço:

```
sudo systemctl edit ollama.service
```

E adicionar as linhas:

```
[Service]
Environment="OLLAMA_NUM_PARALLEL=1"
```

Salve com `Ctrl + O`, aperte `Enter` e saia com `Ctrl + X`. Depois reinicie o serviço:

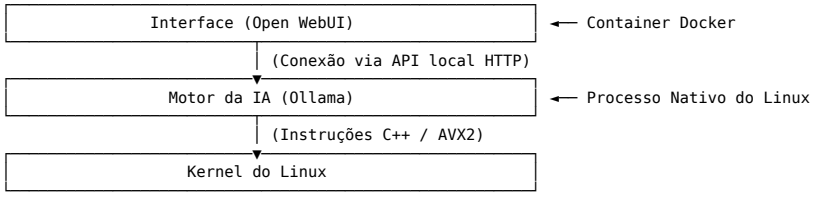
```
sudo systemctl restart ollama
```

User prompt: tenho duvidas esta IA vai rodar direto no kernel ou em um docker? e o meu foco fora ler os PDFs e aprender e me auxiliar em programação isso e possivel.

Response: Onde a IA Roda no Sistema?

A IA **não roda "direto no Kernel"** (o Kernel gerencia o hardware e processos do Linux Mint), mas **também não precisa obrigatoriamente rodar dentro do Docker**.

- **O Ollama (o motor da IA):** Roda nativamente no **User Space** (Espaço do Usuário) como um processo comum em background (`systemd service`), usando binários nativos em C++ executados via instrução AVX2 do seu Ryzen 7.
- **A Interface Web (Open WebUI):** Essa sim é colocada em um container **Docker** apenas por conveniência (facilita não ter que instalar Node.js/Python manualmente).



Se quiser eliminar o Docker completamente para deixar o sistema ainda mais leve, é possível usar aplicativos nativos de interface como o **Pinokio**, **AnythingLLM Desktop** ou **Jan.ai**.

Sim! É Perfeitamente Possível Ler PDFs e Programar

Seu setup com **32 GB de RAM** é excelente para essas duas tarefas específicas.

1. Leitura de PDFs e Documentos (RAG)

Ao usar a interface **Open WebUI** (ou apps nativos como *AnythingLLM*), você pode arrastar e soltar PDFs diretamente na conversa.

- O sistema cria um banco vetorial temporário na sua memória RAM.
- Quando você faz uma pergunta, o modelo faz a busca exata no documento e responde citando as páginas ou trechos correspondentes.

2. Auxiliar em Programação e Estudo

Para código e lógica de software, existem modelos de linguagem treinados especificamente para entender sintaxe, refatoração e *debugging*.

Modelos recomendados para programar no seu Ryzen 7:

- **`qwen2.5-coder:7b`**: Um dos melhores modelos abertos atuais para desenvolvimento de software (gera códigos limpos em Python, C++, JS, Shell Script, etc.).

  ```
  ollama run qwen2.5-coder:7b
  ```

- **`deepseek-coder-v2:16b`** (opcional, para testes): Como você tem 32 GB de RAM, pode testar esse modelo maior. Ele roda com precisão alta para arquitetura de software, embora seja um pouco mais lento no processamento por CPU.

  ```
  ollama run deepseek-coder-v2:16b
  ```

Integração Direta no seu Editor de Código (VS Code)

Além de rodar na interface de chat, você pode integrar o Ollama diretamente ao **VS Code** ou **VSCodium** no Linux Mint para autocomplete de código no seu ambiente local:

1. Instale a extensão **Continue** no VS Code.
2. Nas configurações da extensão, selecione `Ollama` como o provider local.
3. Escolha o modelo `qwen2.5-coder:7b`.

Você terá um copiloto de código totalmente privado, sem enviar nenhuma linha do seu código para servidores externos.

User prompt: ja que posso instalar em um docker o ollama, tem alguma contra indicação?

Response: Instalar **tanto o Ollama quanto a interface em Docker** é uma prática bastante comum, mas quando o assunto é inferência de IA rodando puramente via **CPU/RAM** (como é o caso do seu Ryzen 7 5700U), existem sim algumas contraindicações e gargalos técnicos importantes a considerar:

1. Perda de Desempenho em CPU (Instruções AVX2 e Threads)

- **O Problema:** O motor `llama.cpp` (usado internamente pelo Ollama) depende de compilações altamente otimizadas para as instruções de hardware da sua CPU (AVX2, FMA). Em containers Docker, a abstração da camada de execução e a gestão de threads podem introduzir uma pequena perda de performance na geração de tokens (Tokens por Segundo - TPS).
- **Impacto no seu Ryzen 7:** No Linux nativo, o Ollama acessa diretamente os 16 threads do processador sem nenhum intermediário. No Docker, o gerenciamento de threads e afinidade de CPU pode exigir configurações adicionais nos arquivos de compilação/execução do container.

2. Overhead de Memória RAM (Pequeno, mas Existente)

- **O Problema:** O daemon do Docker e a estrutura da imagem geram um consumo extra de RAM e ciclo de CPU.
- **Impacto no seu setup:** Como você possui **32 GB de RAM**, essa perda de 200 MB a 500 MB de memória é praticamente irrelevante. No entanto, o tempo de carga (*load time*) do modelo da RAM para o processador pode ser ligeiramente mais alto.

3. Armazenamento e Localização dos Modelos (Volume Bind)

- **O Problema:** Modelos de IA ocupam vários gigabytes (o `qwen2.5-coder:7b` tem cerca de 4.7 GB). Se você não mapear um **Volume local** explicitamente na hora de criar o container do Ollama, todos os modelos baixados ficarão armazenados dentro da camada interna do Docker.
- **Impacto:** Se o container for destruído ou recriado, **você perderá todos os modelos baixados** e terá que fazer o download de dezenas de gigabytes novamente.

4. Integração com Outros Programas Locais (Como o VS Code)

- **O Problema:** Quando o Ollama roda nativamente no Linux, ele expõe a API na porta `localhost:11434` de forma automática para todo o sistema.
- **Impacto:** Dentro do Docker, se você não expor as portas corretamente (`-p 11434:11434`) ou não liberar as variáveis de ambiente de CORS (`OLLAMA_ORIGINS="*"`), o plugin do VS Code, scripts em Python ou a interface web não conseguirão se comunicar com a IA.

Resumo e Recomendação de Arquitetura

Para o seu cenário no Linux Mint:

- **Ollama (Motor de IA): Instale Nativo** (`curl -fsSL [https://ollama.com/install.sh](https://ollama.com/install.sh) | sh`). Ele roda como um serviço de sistema (`systemd`), usa 0% de CPU quando ocioso, acessa os 16 threads do Ryzen diretamente e mantém os modelos salvos nativamente na sua pasta `/usr/share/ollama/.ollama` ou `~/.ollama`.
- **Open WebUI (Interface Gráfica): Rode no Docker**. Interfaces gráficas possuem muitas dependências (Node, Python, vetores) e o Docker é perfeito para isolá-las sem poluir seu Linux.

Essa combinação "Híbrida" entrega a **máxima velocidade de inferência na CPU** com a **facilidade de manutenção da interface via Docker**.

User prompt: por favor me faça um passo a paso para realizar a instalação segundo as recomendações, não esquecendo que deve ter o fator ensino e programação como conversamos.

Response: Aqui está o passo a passo completo e organizado para instalar a arquitetura recomendada no seu **Linux Mint** (Ollama nativo para máxima performance da CPU Ryzen 7 + Open WebUI em Docker para a interface + modelos focados em estudo e programação).

Passo 1: Instalar e Otimizar o Ollama (Nativo)

Instalar o motor nativamente garante que o processamento do seu Ryzen 7 5700U acesse as instruções AVX2 diretamente no hardware, sem overhead.

1. Abra o terminal (`Ctrl` + `Alt` + `T`) e execute o instalador do Ollama:

   ```
   curl -fsSL https://ollama.com/install.sh | sh
   ```

2. Ajuste o serviço para aceitar conexões da interface Docker e garantir bom aproveitamento de threads. Abra a edição do serviço:

```
sudo systemctl edit ollama.service
```

3. No editor que abrir, insira o seguinte bloco de texto no topo:

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
Environment="OLLAMA_ORIGINS=*"
```

Pressione `Ctrl + O`, depois `Enter` para salvar, e `Ctrl + X` para sair.

4. Reinicie e aplique as configurações do Ollama:

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

Passo 2: Baixar os Modelos para Programação e Leitura de Documentos

Vamos baixar dois modelos quantizados ideais para a sua RAM de 32 GB e o seu perfil de uso:

1. **Modelo de Programação (Qwen 2.5 Coder 7B):** Especializado em lógica, refatoração e explicação de código em dezenas de linguagens.

```
ollama pull qwen2.5-coder:7b
```

2. **Modelo Educacional e Leitura de PDFs (Phi-3.5 3.8B):** Excelente para síntese de texto, resumos, lógica pedagógica e rápida resposta para chat geral.

```
ollama pull phi3.5
```

(Opcional) Você pode testar se um modelo está rodando diretamente no terminal com `ollama run qwen2.5-coder:7b`. Para sair do chat no terminal, digite `/quit`.

Passo 3: Instalar o Docker no Linux Mint

O Docker servirá exclusivamente para subir a interface gráfica **Open WebUI** sem poluir seu sistema operacional.

1. Instale o Docker e o plugin de execução:

```
sudo apt update
sudo apt install docker.io -y
```

2. Adicione seu usuário ao grupo do Docker para não precisar digitar `sudo` em todo comando:

```
sudo usermod -aG docker $USER
```

3. **Importante:** Encerre a sessão do Linux Mint (Log out) ou reinicie a máquina para que a permissão do grupo Docker seja aplicada ao seu usuário.

Passo 4: Subir a Interface Gráfica (Open WebUI) via Docker

Com a permissão ativa após relogar no Linux:

1. Execute o container da **Open WebUI** conectando-o ao Ollama nativo do seu Linux Mint:

```
docker run -d \
  -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  --name open-webui \
  --restart always \
  ghcr.io/open-webui/open-webui:main
```

2. Abra o seu navegador web (Firefox/Chrome) e acesse:

```
http://localhost:3000
```

3. No primeiro acesso, crie uma conta local (Admin). Esta conta é totalmente privada e salva apenas no banco de dados local da sua máquina.

Passo 5: Usar a Interface para Estudo e Leitura de PDFs (RAG)

Na interface do Open WebUI no navegador:

1. **Escolher o modelo:** No topo da tela de chat, selecione o modelo desejado (ex: `phi3.5` para textos ou `qwen2.5-coder:7b` para programação).
2. **Carregar PDFs e Arquivos:**
   - Na barra de mensagem onde você digita, clique no ícone de **+** ou clique em **Anexar Arquivos**.
   - Escolha o seu PDF de estudo, apostila ou livro.
   - O Open WebUI irá fragmentar o arquivo e criar um índice local de busca rápida.
3. **Fazer perguntas sobre o PDF:** Pergunte algo como: *"Faça um resumo dos pontos principais deste documento"* ou *"Crie 5 perguntas de múltipla escolha com gabarito baseando-se neste texto"*.

Passo 6: Integrar a IA de Programação diretamente no VS Code (Copiloto Local)

Para usar o seu modelo `qwen2.5-coder:7b` como um assistente no VS Code:

1. Abra o **VS Code** (ou VSCodium).
2. Vá até a aba de Extensões (`Ctrl + Shift + X`) e pesquise por **Continue**. Instale a extensão oficial.
3. Clique no ícone da **Continue** que aparecerá na barra lateral esquerda.
4. Escolha a opção de adicionar um novo modelo: selecione **Ollama** como provedor.
5. Indique o modelo `qwen2.5-coder:7b`.
6. Agora você pode selecionar trechos de código com `Ctrl + L` para pedir explicações, correções de bugs ou pedir para o assistente autocompletar funções localmente sem internet.

User prompt: apos fazer o iitem 2 recebi este aviso sudo systemctl edit ollama.service

Response: Esse aviso costuma acontecer porque o arquivo de serviço do Ollama ainda não foi detectado ou você precisa do parâmetro `--full` ou criar o diretório manualmente. No Linux Mint / Ubuntu, o arquivo padrão do serviço é gerado na instalação do script.

Vamos resolver isso de forma direta e sem erro em **2 passos simples**:

Passo 1: Criar a pasta e o arquivo de configuração de parâmetros

Em vez de usar o `systemctl edit`, você pode criar diretamente o arquivo de substituição de configurações (*override*).

Rode este comando no terminal para criar a pasta necessária:

```
sudo mkdir -p /etc/systemctl/system/ollama.service.d
```

Agora, crie e edite o arquivo `override.conf` usando o editor simples **Nano**:

```
sudo nano /etc/systemctl/system/ollama.service.d/override.conf
```

Passo 2: Inserir as configurações e Salvar

Cole o seguinte conteúdo dentro do editor que se abriu no terminal:

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
Environment="OLLAMA_ORIGINS=*"
```

Para salvar e sair no Nano:

1. Pressione `Ctrl + O` e aperte `Enter` para salvar.
2. Pressione `Ctrl + X` para fechar o editor.

Passo 3: Reiniciar o serviço do Ollama

Agora aplique as alterações no sistema com os comandos abaixo:

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

Para confirmar que o Ollama está rodando perfeitamente após a alteração, execute:

```
sudo systemctl status ollama
```

Se aparecer em verde `active (running)`, deu tudo certo! Pode prosseguir para a etapa de **baixar os modelos** (`ollama pull qwen2.5-coder:7b`) e continuar os passos do guia.

User prompt: como verifico se o passo 1 deu certo

Response: Para verificar se o **Passo 1** foi concluído com sucesso e o Ollama está rodando corretamente com as novas configurações no seu Linux Mint, siga estes 3 testes rápidos no terminal:

1. Verificar o Status do Serviço

Execute o comando de status do sistema:

```
systemctl status ollama
```

- **Resultado esperado:** Você deve ver a palavra **`active (running)`** destacada em verde.
- *Para sair dessa tela de status no terminal, pressione a tecla **q**.*

2. Verificar se o Arquivo de Configuração (Override) foi Carregado

Confira se as variáveis de ambiente que configuramos foram lidas pelo serviço:

```
systemctl show ollama --property=Environment
```

- **Resultado esperado:** O terminal deve retornar exatamente a linha:

  ```
  Environment=OLLAMA_HOST=0.0.0.0 OLLAMA_ORIGINS=*
  ```

3. Testar a Resposta da API do Ollama

Faça uma chamada de teste direta para a porta do Ollama:

```
curl http://localhost:11434/
```

- **Resultado esperado:** O terminal deve responder com a mensagem:

  ```
  Ollama is running
  ```

Se os 3 testes acima retornaram os resultados esperados, seu Ollama nativo está 100% pronto e escutando as conexões! Você já pode prosseguir para o **Passo 2** (baixar os modelos com `ollama pull`).

User prompt: no primeiro teste deu Unit ollama.service could not be found.

Response: Isso significa que o script de instalação do Ollama não chegou a criar o serviço do `systemd` no seu Linux Mint.

Não se preocupe, vamos corrigir isso rapidamente em **3 etapas**:

Passo 1: Verificar se o executável do Ollama já está instalado

Rode no terminal:

```
ollama --version
```

- **Se retornar a versão (ex: `ollama version is 0.x.x`):** O binário está instalado, apenas falta criar o arquivo do serviço. Vá para o **Passo 2**.
- **Se retornar `command not found`:** O script oficial de instalação não foi concluído. Execute o comando de instalação novamente:

  ```
  curl -fsSL https://ollama.com/install.sh | sh
  ```

Passo 2: Criar o arquivo de serviço do Ollama manualmente

Como o serviço não foi encontrado, vamos criar o arquivo de serviço do zero no local padrão do sistema (`/etc/systemd/system/ollama.service`).

1. Crie um novo usuário de sistema dedicado ao Ollama (boa prática de segurança no Linux):

```
sudo useradd -r -s /bin/false -m -d /usr/share/ollama ollama
```

2. Crie o arquivo de serviço usando o editor nano:

```
sudo nano /etc/systemd/system/ollama.service
```

3. Cole o seguinte conteúdo dentro do arquivo:

```
[Unit]
Description=Ollama Service
After=network-online.target

[Service]
ExecStart=/usr/local/bin/ollama serve
User=ollama
Group=ollama
Restart=always
RestartSec=3
Environment="OLLAMA_HOST=0.0.0.0"
Environment="OLLAMA_ORIGINS=*"

[Install]
WantedBy=default.target
```

4. Para salvar e sair:
   - Aperte `Ctrl + O` e dê `Enter`.
   - Aperte `Ctrl + X` para fechar.

Passo 3: Ativar e iniciar o serviço

Agora diga ao Linux Mint para reconhecer, ativar na inicialização do sistema e rodar o Ollama:

```
sudo systemctl daemon-reload
sudo systemctl enable --now ollama
```

Passo 4: Fazer o teste novamente

Rode o comando de verificação:

```
systemctl status ollama
```

Agora deve aparecer a mensagem **`active (running)`** em verde! Pressione a tecla q para sair da tela de status e você estará pronto para rodar `ollama pull qwen2.5-coder:7b`.
